Training a coding agent through a harness you did not write

Sergio Paniego | **@sergiopaniego** 🤗 Hugging Face

Kernel Panic, AI Open Models Conference
Madrid, 06 Oct 2026

THE SAME WEIGHTS, ON THE SAME TASKS

62% Mini-SWE-Agent

vs

33% Claude Code

The only difference is the program wrapped around the model.

LFM2.5-2.6B, 250 held-out SmolDataEnvs tasks, before any training

01 / THE VOCABULARY

An agent is a model plus a harness

Claude Code · Codex · OpenCode · Cursor · Cline · Pi · Goose · Mini-SWE-Agent

runs the loop

it calls the model again and again, and decides when to stop

picks the tools

the model only gets the tools this harness exposes, under its names

writes the context

what the model reads is assembled, trimmed and compacted by the harness

reads the reply

and rejects it if the format is not the one it expects

Change the harness, and you change what the model sees and may do.

None of them was written with training in mind, and we do not change a line of their code.

Every harness speaks its own dialect

Harness	Anthropic Messages	OpenAI Responses	Google generateContent	chat completions	plain text	Tool calls
Claude Code	●					yes
Codex		●				yes
OpenCode	●	●	●	●		yes
Goose	●		●	●		yes
SWE-agent				●		yes
Mini-SWE-Agent		○		●	○	one tool
OpenHands SDK				●		yes
Aider				●		no
Qwen Code			●	○		yes
Kimi CLI				●		yes
Terminus 2				●		no
Hermes			*three APIs, switchable*			yes

LEGEND

● speaks it by default ○ also supports it

This is how frontier labs train coding models today

In 2025 most model reports did not name a harness at all. These are all 2026.

Kimi K3	builds its harnesses from modules, trains across them
Qwen3-Coder-Next	agentic coding data from six harnesses
MiMo-V2.6	a pool of task-adapted mini-harnesses
Poolside Laguna	1.3B tokens of OpenHands, OpenCode, Mini-SWE-Agent
Liquid LFM2.5	a harness picked at random per task

Leaving the harness you trained in can end the run

Orchard ran finished coding models under three harnesses on the same tasks. One of them, Kimi-CLI, was new to every model in the comparison.

degraded
the model still works, and solves less

OpenSWE-32B goes from 62.4% at home to 54.9% in Mini-SWE-Agent, and 3.6% under Kimi-CLI. Zero on Terminal-Bench 2.0.

catastrophic
the output stops being usable at all

Scale-SWE stops producing valid tool calls anywhere but the harness it was trained in. Those cells have no number to report.

Orchard, SWE-bench Verified and Terminal-Bench 2.0, same tasks under every harness

05 / THE CAUSE

Three ways a model overfits to its harness

	trained in a tool-calling harness	dropped into aider, which takes edits in prose
Format anchored to one action format	`"tool_calls": [{"name": "edit_file", …}]`	`<<<<<<< SEARCH … ======= … >>>>>>> REPLACE`
Context structure anchored to one way of stacking history	`role: "tool"` → one message per result	`role: "user"` → one summarised turn
Control flow anchored to someone else's retry and stop rules	`exit 1` → harness retries, then stops at step N	`exit 1` → error returned to the model

WHAT YOU SEE

Format: The JSON arrives as chat text. Nothing is parsed, nothing is edited, and the turn is scored as a non-answer.

Context structure: Its own earlier results are no longer where it expects them, so it repeats work it has already done.

Control flow: It runs the same failing command again, because deciding when to stop was never its job.

The three modes are from the KAT-Coder report. One fix covers all three: vary the harness during training.

Open weights, closed training surface

WHAT YOU GET

+ Open weights, often a strong model
+ A benchmark table
+ A paragraph naming the harnesses

WHAT YOU DO NOT

- The harness pool, wired the way they ran it
- The environment mix, and how it was sampled
- The capture layer that turns a harness loop into training samples
- The trainer wiring on the other side of it

SO WHAT IS MISSING

An open way to train inside any harness, without touching its code

Agentic RL, in four steps

1. Give the model a task, and let it try the same one several times
2. Grade each attempt with a number. Did the tests pass?
3. Compare the attempts against their own average
4. Make the better-than-average tokens more likely, the worse ones less

That is **GRPO**. The group is its own baseline, so you never need a second value model. Agentic just means each attempt is a whole session with tools, with a filesystem and a shell behind it.

An RL environment is where it practises



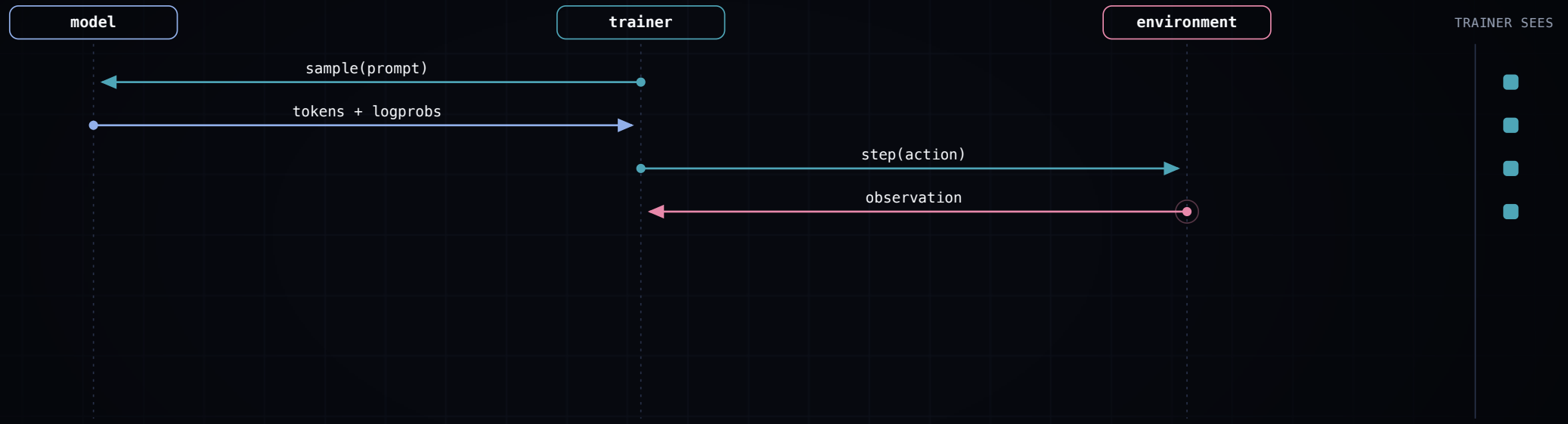
A coding harness already has every one of those.

state	←	the filesystem the agent works in
actions	←	its own write, bash and edit tools
reward	←	tests that either pass or do not

A coding harness was already an environment.

Who drives the loop: the trainer or the harness

White box | Black box



4/4
EXCHANGES THE TRAINER STARTS

8/8
EVENTS THE TRAINER OBSERVES

THE TRAINER DRIVES IT
WHO OWNS THE LOOP

ARROW COLOUR IS WHOEVER MADE THE CALL

model — trainer — environment — dashed: the trainer is not there

The trainer starts every exchange, so it sees the whole episode.

Agent Lightning calls this harnessed agentic RL. The harness owns the loop, and the trainer only sees the model's requests and responses. Agent Lightning, Microsoft 2026

OpenEnv is the shared interface

What it is	a standard interface for RL environments: Gymnasium-style reset, step and state, over a client and server transport, with typed actions and observations
How it ships	packaged as Docker, publishable to the Hub, so an environment is a repo you can pull
Who steers it	started as a Meta PyTorch and Hugging Face collaboration, now a committee of twelve organisations, BSD-3

It does not train anything and it does not define rewards. It is what the harness, the environment and the trainer all plug into, and it is where the **capture proxy** lives.

hf.co/docs/openenv v0.7.0 or later

TRL is the trainer

Hugging Face’s post-training library. One trainer class per method, same shape for all of them.

SFTTrainer | DPOTrainer | GRPOTrainer | DistillationTrainer | **AsyncGRPOTrainer**

AsyncGRPOTrainer

Generation and training run apart. One GPU trains while a second serves the policy with vLLM and keeps producing rollouts, so training never waits for the slowest agent. The price is that rollouts come from weights usually two optimizer steps old, and never more than four.

hf.co/docs/trl examples/async_grpo_harbor

harbor brings the tasks and the sandboxes

40+
harness adapters

10
validated end to end in OpenEnv

1
fresh sandbox per rollout

Harbor runs agents against containerised tasks, from the team behind Terminal-Bench. A harness needs a task and a box to run it in, and wiring that yourself means writing the glue for every harness against every sandbox backend. Harbor keeps the task, the harness and the sandbox independent of each other, so each one becomes something you pick per rollout.

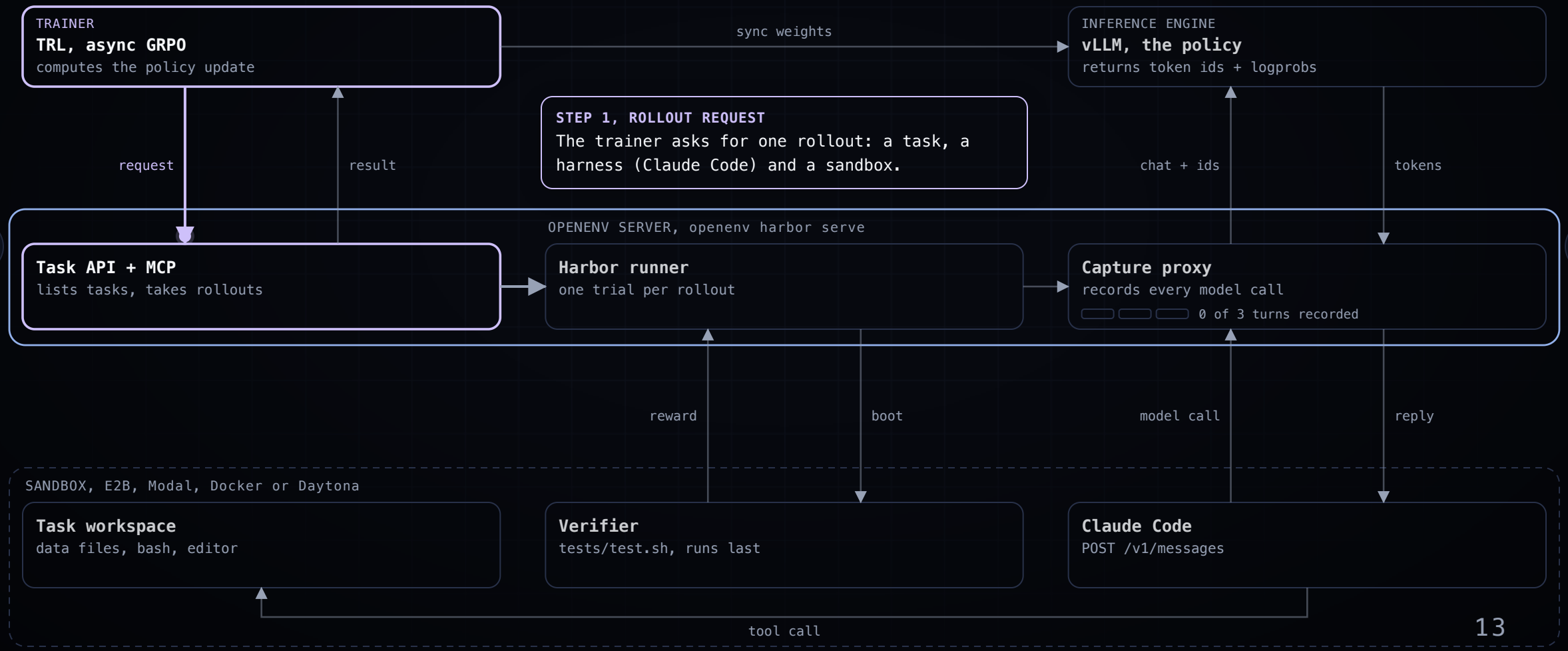
```
env = HarborEnv(task=..., harness="claude-code")
```

harborframework.com

One rollout, end to end

Pause | ‹ Back | Next › | Step 1 of 21

Harness: **Claude Code** | Codex | OpenCode



The capture proxy

Pause **2/11** the harness calls what it thinks is a model endpoint, in its own dialect



CAPTURE STORE prompt, masked sampled, one logprob each 0 trainable tokens, turn 1

turn 1 waiting for the first completion

The proxy probes the engine before it trusts it. An engine that will not return token ids and logprobs still serves the rollout, and the trace comes back marked evaluation only. Every hosted API lands there.

SO WE RAN IT

LFM2.5-2.6B, four harnesses, 1,000 steps

OpenCode Claude Code Codex Mini-SWE-Agent

The tasks: SmolDataEnvs

5,000 training tasks **250** held-out, harder on purpose **471** Kaggle datasets built from real Kaggle notebooks, through the jupyter-agent dataset

Easy, level 1 | **Medium, level 2** | Hard, level 4

What percentage of all matches have a goal difference of zero (i.e., draws)? Gold answer: 25.4% data from hugomathien/soccer

tasks/0000_369_369503_qa_1/

- task.toml — the spec
- instruction.md — the prompt
- environment/
  - Dockerfile — the image
  - pull_bucket.py — the data
- tests/
  - test.sh — the verifier
  - grader.py — the reward

instruction.md **The prompt.** Exactly what the agent is shown: where its files are, what is installed, and how to write the answer. this task only

```
1  You are a data-analysis agent working in a sandbox. Use your code-execution tool to inspect the files and compute the
   answer.
2
3  Files (in /home/user/input, no subfolders):
4  - database.sqlite
5
6  Installed: pandas, numpy, matplotlib, seaborn, scipy, scikit-learn, statsmodels, tabulate, sqlite3, plotly (pip install
   more if needed).
7
8  Question:
9  What percentage of all matches have a goal difference of zero (i.e., draws)?
10
```

Verified by a checker, not judged by a model. Every task had to be solved in a live sandbox before it was kept, and that test script decides the reward. No model sits in the reward path.

The setup

model	LFM2.5-2.6B, trained by Liquid AI in Hermes Agent and OpenClaw, neither of them ours
tasks	1,000 of those training tasks, 400 medium and 600 hard, in one fixed order
test	250 held-out tasks, each run under all four harnesses, 1,000 cells in all
two runs	one in OpenCode alone, one rotating the four across GRPO groups
recipe	async GRPO in TRL, 1,000 steps, 2x H100, one E2B sandbox per rollout
time	32 h for OpenCode-only, 46 h for multi-harness

LiquidAI/LFM2.5-2.6B FineEnvs/SmolDataEnvs

Both runs follow the same task order, and both are evaluated the same way, including under the three harnesses the OpenCode-only run never saw.

What we rewarded

correctness
1 for a correct answer, 0 for a wrong one. Graded by the task's own verifier.

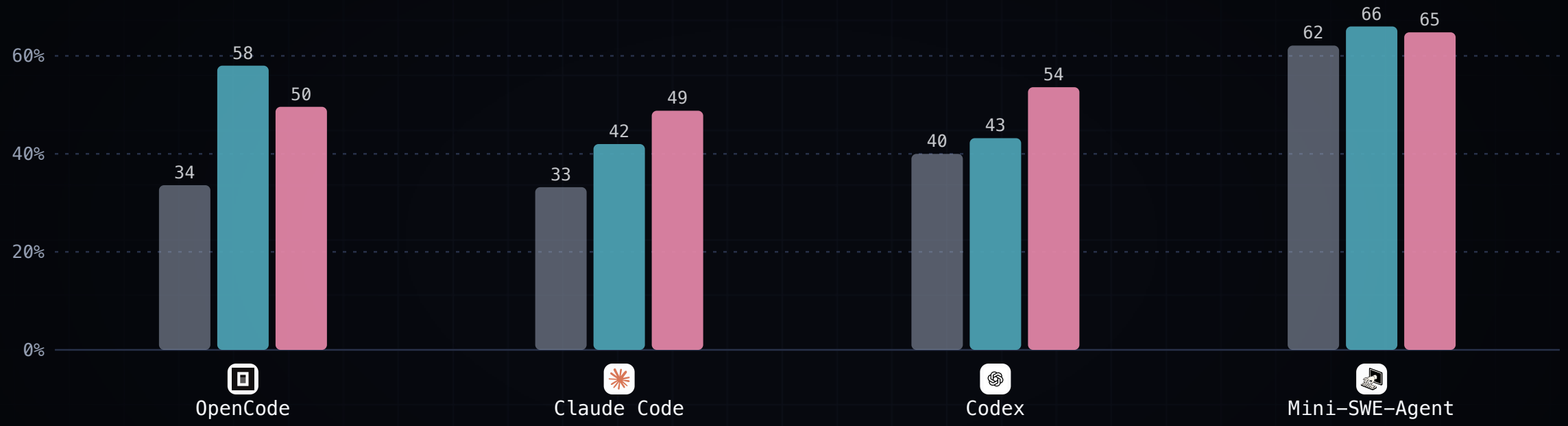
tool efficiency
at most 0.1, and only on a correct answer, for taking fewer calls.

The bonus is tiny, and it is the reason the run works. GRPO learns from differences **inside a group.** When all eight rollouts are correct, correctness alone makes them identical and the group teaches nothing.

22.6% of multi-harness groups were all-correct with different tool counts.

For those, the bonus was the only thing telling the eight rollouts apart.

Where each run ends up, by harness



LEGEND

LFM base model LFM, OpenCode only LFM, multi-harness

Pass@1 at step 1,000 under each harness, next to the base model.

TRAINED ACROSS ALL FOUR

42% → 54%

BEFORE AFTER

of 250 held-out tasks solved, averaged over the four harnesses

31%
fewer tool calls

4 / 4
harnesses improved

Trained in OpenCode alone it reaches 58% there, and gains much less everywhere else.

Fewer calls, and fewer tokens with them

Tool calls | Output tokens | Input tokens

Trained in OpenCode only

step	OpenCode	Claude Code	Codex	Mini-SWE-Agent
100*	−26%	+1%	+3%	−1%
200*	−30%	+10%	−2%	+2%
300*	−31%	−2%	−7%	+2%
400*	−25%	+2%	−7%	−2%
500	−33%	+5%	−11%	−1%
600	−35%	−9%	−15%	−4%
700	−32%	−7%	−17%	+1%
800	−37%	−6%	−24%	0%
850				
900	−35%	−7%	−18%	−3%
1,000	**−36%**	**+10%**	**−24%**	**−2%**

Trained across four harnesses

step	OpenCode	Claude Code	Codex	Mini-SWE-Agent
100*	−23%	−3%	−15%	−6%
200*	−32%	−10%	−20%	−7%
300	−31%	−12%	−33%	−7%
400	−24%	−19%	−41%	−13%
500	−29%	−20%	−41%	−13%
600	−32%	−16%	−44%	−12%
700	−31%	−19%	−47%	−15%
800	−23%	−18%	−50%	−11%
850	−25%	−16%	−50%	−16%
900	−35%	−18%	−52%	−15%
1,000	**−33%**	**−28%**	**−53%**	**−16%**

fewer than the base model −50% 0 +50% more

Each cell is the change in tool calls against the base model, on the test tasks both solved under that harness. −26% means 26% fewer. Blue is fewer, orange is more. * marks an evaluation that was still missing a few cells.

What this does not prove

! One task family, data analysis. Nothing here says it carries to other work.

! Every run is a single seed. One model at 2.6B, 1,000 steps.

! The two runs did not even see the same amount of data, 162M tokens against 451M.

! Averaged over the four harnesses, training on all four and training on OpenCode alone come out about level. These runs show where each one gains, they do not crown one of them.

We are setting up larger runs now, with bigger models on the same stack.

The harness is half of it. The environments are the other half

The same labs that vary the harness also train over large pools of environments at once. Nothing in this stack is tied to data analysis, so that half is open too.

MiMo-V2.6-RL	7,780 of Xiaomi's own RL environments as Harbor tasks: code, cyber, terminal, webdev, music, general
SmolDataEnvs	5,000 data-analysis tasks, the ones behind every number in this talk
Repo2RLEnv	verifiable coding and terminal environments generated from real repositories
Single-skill envs	GeoGuesser, LaTeX-OCR, paint-with-code, Wordle, each one a Space you can open right now

You pick the harness and you pick the environments, and the trainer stays where it is.

Run it yourself

01	**Install**	openenv[harbor] on Python 3.12, and TRL from main until the worker ships
02	**Pick a task set**	any Harbor dataset on the Hub, ours or your own
03	**Pick the harnesses**	one string each, rotated per rollout
04	**Run the example**	examples/async_grpo_harbor, unchanged

the example OpenEnv docs the tutorials

A harness you did not write is now somewhere you can train.

Everything, in one place

READ

- The article
- Why Harbor exists
- RL environments guide
- RL environments 101
- FineEnvs Academy
- The LFM2.5-2.6B release

RUN

- Everything in one collection
- TRL Harbor example
- The tutorials
- The environment, live
- Harbor task visualiser
- Every training curve

MODELS AND DATA

- Multi-harness RL model
- OpenCode-only RL model
- The SFT models
- The base model
- SmolDataEnvs
- MiMo-V2.6 RL environments

DOCS AND CODE

- TRL docs
- Async GRPO trainer
- OpenEnv docs
- Harbor
- TRL on GitHub
- OpenEnv on GitHub

The deck is at hf.co/spaces/FineEnvs/multi-harness-rl-slides

Sergio Paniego **@sergiopaniego** Hugging Face

THESE SLIDES